



UNIVERSITATEA DIN
BUCUREȘTI

FACULTATEA DE
MATEMATICĂ ȘI
INFORMATICĂ



SPECIALIZAREA INFORMATICĂ

Lucrare de licență

ARHITECTURA PAN CA MODEL PENTRU APLICAȚII WEB FULL-STACK

Studiu de caz privind reducerea risipei alimentare

Absolvent: Eduard - Emanuel Dinea

Coordonator științific: Lect. dr. Mihai Cherciu

București, iulie 2026

Rezumat

Rezumat

Această lucrare prezintă proiectarea și implementarea unei aplicații web full-stack, denumite *Kookio*, realizată ca studiu de caz pentru analizarea arhitecturii **PAN** (Prisma–Analog–Nx) în contextul aplicațiilor web moderne. Aplicația are ca scop gestionarea inventarului de ingrediente pentru utilizatori casnici, contribuind indirect la reducerea risipei alimentare printr-o mai bună organizare a resurselor disponibile.

Motivația lucrării pornește de la necesitatea utilizării unor arhitecturi web care să ofere performanță ridicată, coerență între frontend și backend și o experiență de dezvoltare unitară. În acest context, lucrarea investighează arhitectura PAN ca model full-stack bazat pe ecosistemul TypeScript, în care straturile aplicației sunt integrate într-un flux comun de dezvoltare.

Aplicația este realizată folosind randare server-side cu Angular prin framework-ul Analog, comunicare type-safe între client și server prin tRPC și persistența datelor cu Prisma și CockroachDB. Organizarea proiectului într-un monorepo Nx permite modularitate, testare automată și execuție incrementală a task-urilor.

Implementarea este validată prin testare automată unitară și end-to-end, utilizând Vitest și Playwright, precum și prin rularea aplicației într-un mediu de deployment modern. Rezultatele obținute demonstrează că arhitectura PAN reprezintă un model viabil pentru dezvoltarea aplicațiilor web full-stack scalabile, coerente și extensibile.

Cuprins

Capitolul 1

Introducere

Motivație

În contextul vieții moderne, tot mai multe persoane caută soluții digitale care să simplifice organizarea meselor zilnice și gestionarea ingredientelor disponibile în gospodărie. Lipsa unei evidențe clare a ingredientelor conduce frecvent la achiziții redundante și la risipă alimentară, fenomen cu impact atât economic, cât și social.

În același timp, aplicațiile web moderne se confruntă cu provocări tehnice semnificative, precum performanța la încărcare, coerența dintre frontend și backend și mentenanța pe termen lung a codului. Aceste provocări devin mai accentuate în aplicațiile full-stack, unde integrarea deficitară a straturilor conduce la creșterea complexității și a costurilor de dezvoltare.

Proiectul *Kookio* abordează aceste probleme prin adoptarea unei arhitecturi web moderne, bazate pe ecosistemul **TypeScript**, care integrează frontend-ul, backend-ul și stratul de persistență într-un flux unitar de dezvoltare. În acest context, este analizată arhitectura **PAN (Prisma–Analog–Nx)** ca model pentru realizarea unor aplicații web full-stack coerente, extensibile și ușor de testat.

Scopul lucrării

Scopul acestei lucrări este proiectarea și implementarea unei aplicații web full-stack care permite gestionarea inventarului de ingrediente pentru utilizatori casnici, servind drept studiu de caz pentru evaluarea arhitecturii PAN și pentru explorarea impactului acesteia asupra dezvoltării aplicațiilor moderne.

În acest sens, lucrarea își propune:

- realizarea unui sistem de gestiune a ingredientelor, cu persistență într-o bază de date relațională;
- integrarea frontend-ului și backend-ului într-un flux de dezvoltare unitar, cu partajarea tipurilor și a validărilor;

- analiza beneficiilor arhitecturii PAN în ceea ce privește modularitatea, productivitatea și mentenanța;
- validarea implementării prin testare automată unitară, de integrare și end-to-end.

Aplicația este concepută ca un produs minim viabil (MVP), care poate fi extins ulterior cu funcționalități precum recomandarea de rețete sau integrarea cu furnizori externi, contribuind astfel la reducerea risipei alimentare. Detaliile privind implementarea sunt prezentate în Capitolul ??.

Capitolul 2

Preliminarii

Contextul tehnologic

Proiectul *Kookio* se bazează pe arhitectura unitară **PAN** (Prisma–Analog–Nx), care integrează fluxul complet de dezvoltare **TypeScript** de la interfață până la bazele de date. Această arhitectură facilitează modularitatea, testarea automată și extinderea sistemului, fiind susținută de instrumente moderne pentru orchestrare, randare hibridă și persistență tipizată [9, 6, 15]. O prezentare detaliată a manifestului, a avantajelor comparative și a direcțiilor de evoluție este centralizată în **Anexa ??**.

2.1 Preliminarii

2.1.1 Arhitecturi web moderne

Aplicațiile web moderne au evoluat semnificativ față de modelul clasic client–server, în care browserul funcționa exclusiv ca un consumator de API-uri REST. Cerințele actuale privind performanța, experiența utilizatorului și optimizarea pentru motoarele de căutare au condus la apariția unor arhitecturi hibride, care combină randarea pe client cu randarea pe server [6, 10].

Printre aceste abordări se numără:

- **Client-Side Rendering (CSR)**, unde aplicația este randată integral în browser;
- **Server-Side Rendering (SSR)**, unde conținutul HTML este generat pe server pentru fiecare cerere;
- **Static Site Generation (SSG)**, unde paginile sunt generate la build-time;
- **arhitecturi hibride**, care combină aceste tehnici în funcție de tipul conținutului.

În contextul proiectului Kookio, este adoptată o arhitectură full-stack modernă, care integrează frontend-ul și backend-ul într-un flux unitar de dezvoltare, permițând partajarea tipurilor și a logicii de validare între straturi [16]. Această abordare reduce complexitatea integrării și crește coerența aplicației.

2.1.2 Angular și randare server-side

Angular este un framework matur pentru dezvoltarea aplicațiilor web, bazat pe TypeScript și orientat către arhitecturi scalabile [7]. În mod tradițional, Angular a fost utilizat predominant în scenarii de tip client-side rendering, însă versiunile recente extind semnificativ suportul pentru randare server-side.

În cadrul proiectului Kookio, Angular este utilizat împreună cu Analog, un meta-framework care adaugă suport nativ pentru:

- Server-Side Rendering (SSR),
- rutare bazată pe fișiere,
- integrarea logicii de server în aceeași aplicație [6].

Randarea server-side permite generarea conținutului HTML direct pe server, îmbunătățind timpul de afișare inițială și compatibilitatea cu motoarele de căutare [10]. Această caracteristică este esențială pentru aplicații orientate către conținut, precum Kookio, unde lizibilitatea inițială și performanța percepută sunt importante.

Analog facilitează această abordare fără a separa artificial frontend-ul și backend-ul, păstrând o bază de cod unificată și coerentă.

2.1.3 Monorepo și orchestrare cu Nx

Proiectul este organizat sub forma unui monorepo, utilizând instrumentul Nx. Abordarea monorepo permite gestionarea mai multor aplicații și biblioteci într-un singur repository, cu reguli explicite de dependență și caching la nivel de task [9].

În cadrul proiectului Kookio, această structură facilitează separarea clară între:

- aplicația web (frontend și server-side logic),
- stratul de acces la date (Prisma Client),
- validarea și contractele de date (Zod),
- utilitare și date mock partajate.

Nx oferă avantaje importante precum:

- execuția incrementală a task-urilor (build, test, lint),
- cache local și determinism al build-urilor,
- izolare logică între proiecte prin reguli de boundary [9].

Această abordare este deosebit de potrivită pentru aplicații full-stack moderne, unde frontend-ul și backend-ul evoluează în paralel și partajează tipuri și contracte comune.

2.1.4 tRPC și partajarea tipurilor

Pentru comunicarea între client și server, proiectul utilizează tRPC, un framework care permite definirea endpoint-urilor API într-un mod complet tipat, fără a necesita generarea manuală de clienți sau specificații intermediare [16].

În această arhitectură, procedurile tRPC sunt definite pe server și consumate direct în aplicația Angular, beneficiind de:

- inferență completă a tipurilor TypeScript,
- validare automată a input-urilor prin Zod [2],
- eliminarea discrepanțelor dintre contractele client și server.

Această abordare reduce semnificativ erorile de integrare și accelerează dezvoltarea, fiind adecvată în special aplicațiilor full-stack TypeScript.

2.1.5 Persistență: Prisma și CockroachDB

Stratul de persistență este construit folosind Prisma ca ORM și CockroachDB ca sistem de gestiune a bazei de date [15, 3]. CockroachDB este un sistem NewSQL distribuit, compatibil cu protocolul PostgreSQL, care oferă consistență tranzacțională și scalabilitate orizontală [14].

Prisma este utilizat pentru:

- definirea schemei bazei de date într-un limbaj declarativ,
- generarea automată a clientului TypeScript,
- gestionarea migrațiilor între versiuni ale schemei.

Alegerea acestei combinații permite proiectului să rămână compatibil cu un ecosistem SQL matur, dar și pregătit pentru scenarii de scalare viitoare, fără modificări semnificative ale codului aplicației.

2.1.6 Testare automată: Vitest și Playwright

Testarea automată reprezintă un element esențial în dezvoltarea aplicațiilor moderne, asigurând corectitudinea funcțională și stabilitatea în timp. În cadrul proiectului Kookio sunt utilizate două tipuri complementare de testare: teste unitare și teste end-to-end.

Pentru testarea unitară și de integrare este utilizat Vitest, un framework de testare rapid, compatibil cu ecosistemul Vite și TypeScript [12, 11]. Vitest permite testarea componentelor Angular, a utilităților și a logicii de business într-un mediu izolat, cu suport pentru mocking și rulare incrementală.

Testarea end-to-end este realizată cu ajutorul Playwright, care permite simularea interacțiunilor reale ale utilizatorului în browser [8]. Acest tip de testare validează

fluxurile critice ale aplicației, precum încărcarea paginilor, interacțiunea cu formularul și comunicarea cu API-ul server-side.

Combinarea celor două abordări oferă o acoperire echilibrată:

- Vitest validează corectitudinea logicii interne,
- Playwright verifică comportamentul aplicației din perspectiva utilizatorului final.

Această strategie contribuie la creșterea încrederii în stabilitatea aplicației și facilitează evoluția incrementală a codului.

2.1.7 Managementul configurației și mediilor

Variabilele de mediu sunt gestionate centralizat în platforma de deployment Vercel, care acționează ca sursă unică de adevăr pentru configurația aplicației. Această abordare permite separarea clară a mediilor (dezvoltare, testare, producție) și reduce riscul expunerii accidentale a secretelor [18].

Pentru rularea locală, Vercel CLI este utilizat pentru a sincroniza temporar variabilele de mediu active prin comanda `vercel env pull`. Astfel, mediul local rămâne aliniat cu mediul de deploy, fără a necesita fișiere `.env` persistente în repository.

Capitolul 3

Implementare

3.1 Prezentare generală a implementării

Aplicația *Kookio* este implementată conform arhitecturii **PAN**, descrisă în Anexa ??
Această secțiune prezintă structura concretă a proiectului, organizarea codului și fluxurile principale de execuție, din perspectiva implementării efective.

3.1.1 Structura monorepo-ului Nx

```
kookio/
├── web/                                # aplicație Analog SSR Angular 20
│                                       cu client tRPC și module funcționale
├── web-e2e/                           # suită de teste Playwright pentru
│                                       aplicația web
├── libs/
│   ├── data-access/
│   │   ├── prisma-client              # schemă Prisma, migrații și
│   │   │                               scripturi dependente de mediu
│   │   └── prisma-validator           # scheme Zod generate din Prisma,
│   │                                       DTO-uri și validări suplimentare
│   └── shared/
│       ├── mock/                      # date de tip mock (ex. mockIngredients)
│       │                               pentru demonstrații și seed
│       └── utils/                     # utilitare TypeScript reutilizabile
│                                       între aplicații
├── dist/                              # artefacte de build generate de Nx
├── nx.json                            # configurație globală Nx și
│                                       valori implicite pentru target-uri
├── package.json                       # manifestul principal al proiectului
├── tsconfig.base.json                 # configurația de bază TypeScript
├── vercel.json                        # configurația proiectului Vercel
└── vitest.workspace.ts                # configurația Vitest la nivel de
                                        workspace
```

Această organizare permite separarea clară a responsabilităților între aplicația web, testele end-to-end și bibliotecile reutilizabile, facilitând dezvoltarea incrementală și testarea independentă a componentelor [9].

3.1.2 Fluxul aplicației end-to-end

În implementarea aplicației Kookio, fluxul principal de date urmează traseul: *Analog (UI și SSR) → tRPC (API type-safe) → Prisma (acces la date)* [6, 16, 15].

Această structură permite propagarea tipurilor între client și server și integrarea directă a logicii de business în cadrul rutelor server-side [16].

3.2 Implementarea aplicației

Acest capitol descrie implementarea practică a aplicației Kookio, evidențiind modul în care conceptele prezentate în capitolele anterioare sunt transpuse într-un sistem funcțional. Sunt analizate structura proiectului, organizarea codului, fluxurile de date și deciziile tehnice care au stat la baza realizării aplicației.

3.2.1 Arhitecturi web moderne

Aplicarea practică a arhitecturilor web moderne presupune alegerea unei strategii de randare adaptate tipului de conținut și cerințelor de performanță. În proiectul *Kookio*, randarea este abordată **hibrid**, combinând randarea server-side cu generarea de pagini statice acolo unde este posibil [6, 10].

- **SSR (Server-Side Rendering)** este utilizat pentru paginile dinamice, unde conținutul trebuie livrat rapid și în mod consistent indiferent de dispozitiv, serverul generând HTML-ul inițial [6, 10].
- **SSG (Static Site Generation / prerender)** este utilizat pentru rute stabile, care pot fi pre-generate la build-time (de exemplu pagini informative), reducând costul de execuție la runtime [6].
- **CSR** rămâne relevant prin faptul că, după încărcarea inițială, interfața funcționează ca o aplicație interactivă în browser, iar actualizările sunt realizate fără reîncărcarea completă a paginii.

Această combinație permite un compromis eficient între performanță, flexibilitate și experiența utilizatorului, fără a separa artificial aplicația în proiecte distincte de frontend și backend [6, 10].

3.2.2 Structura proiectului și organizarea codului

Aplicația Kookio este implementată într-un monorepo gestionat cu Nx, care permite organizarea coerentă a aplicațiilor și bibliotecilor reutilizabile [9]. Această abordare facilitează separarea responsabilităților și încurajează reutilizarea codului, menținând în același timp un control centralizat asupra configurațiilor și dependențelor.

Structura principală a proiectului este împărțită în:

- o aplicație web principală, responsabilă de interfața utilizator și logica server-side;
- biblioteci de acces la date, care conțin schema bazei de date și codul asociat ORM-ului [15];
- biblioteci partajate, utilizate pentru utilitare și date mock.

Această organizare permite dezvoltarea incrementală a aplicației și extinderea facilă a funcționalităților, fără a introduce dependențe circulare sau redundanță [9].

3.2.3 Angular și randare server-side

În cadrul proiectului *Kookio*, interfața este construită cu Angular, iar suportul pentru randare server-side este obținut prin meta-framework-ul **Analog**, care rulează peste runtime-ul **Nitro** [6, 10]. Această integrare permite ca aplicația să livreze HTML inițial generat pe server, iar ulterior să devină complet interactivă în browser.

Fluxul de execuție este compus din două etape:

- **Randare pe server (SSR)**: la accesarea unei rute, serverul generează HTML-ul inițial și îl trimite către client, reducând timpul până la primul conținut vizibil și îmbunătățind compatibilitatea cu motoarele de căutare [6, 10].
- **Activarea interfeței în client**: după încărcare, codul Angular rulează în browser și preia controlul asupra interfeței, permițând interacțiuni dinamice și apeluri către API fără reîncărcări complete.

Un avantaj important al acestei abordări este faptul că rutele de aplicație și rutele de tip API coexistă în același proiect (ex.: endpoint-uri de tip tRPC și rute auxiliare precum `/api/v1/health`), permițând implementarea unui sistem full-stack coerent, cu acces la baza de date realizat exclusiv pe server [6, 10, 16].

3.2.4 Gestionarea datelor cu Prisma și CockroachDB

Persistența datelor este realizată cu ajutorul Prisma ORM, configurat pentru a utiliza CockroachDB prin protocolul PostgreSQL [15, 3, 14]. Prisma oferă un model declarativ pentru definirea structurii bazei de date, precum și un client tipizat pentru interogarea acesteia din aplicație [15].

Schema bazei de date este definită într-un fișier central, din care Prisma generează automat:

- structura tabelelor și a relațiilor,
- clientul de acces la date,
- tipuri TypeScript asociate modelelor [15].

Pentru modelul **Ingredient**, este utilizată o constrângere de unicitate compusă, care asigură faptul că un utilizator nu poate avea ingrediente duplicate cu același nume. Această regulă este exprimată direct în schema Prisma și este aplicată automat la nivelul bazei de date [15].

3.2.5 Comunicarea client–server cu tRPC

Pentru comunicarea dintre client și server este utilizat tRPC, un framework care permite definirea API-urilor tipizate fără a introduce un strat suplimentar de serializare manuală [16]. Prin această abordare, tipurile sunt partajate între frontend și backend, eliminând discrepanțele dintre contractele de date [16].

Routerele tRPC sunt definite pe server și expun operații precum:

- crearea unui ingredient,
- listarea ingredientelor,
- actualizarea și ștergerea acestora.

Clientul Angular consumă aceste operații direct, beneficiind de verificare statică a tipurilor și de autocompletare în timpul dezvoltării [16]. Această integrare reduce semnificativ riscul de erori la rulare și crește productivitatea dezvoltatorului.

3.2.6 Randare server-side și integrarea backend-ului

Aplicația utilizează randare server-side prin Analog, care rulează peste infrastructura Nitro [6, 10]. Această abordare permite definirea rutelor API și a logicii de server direct în cadrul aplicației web, fără a necesita un backend separat [10].

Rutele server-side includ atât endpoint-uri de tip API, cât și handler-e pentru tRPC [16, 10]. Prin această integrare, aplicația poate executa interogări către baza de date direct pe server, în contextul fiecărei cereri HTTP [15].

Această arhitectură reduce latența percepută de utilizator și permite un control mai bun asupra fluxului de date și al securității aplicației.

3.2.7 Logare și observabilitate

Pentru monitorizarea comportamentului aplicației și facilitarea depanării, *Kookio* utilizează un mecanism de logare unitar, disponibil atât în context server-side, cât și în execuția locală.

Logarea este realizată cu ajutorul bibliotecii **Pino** [1], un logger performant pentru aplicații Node.js, ales pentru impactul minim asupra performanței și integrarea facilă în medii server-side moderne. Această abordare permite înregistrarea coerentă a evenimentelor relevante din fluxul de execuție, fără a afecta semnificativ performanța aplicației.

3.2.8 Fluxul de execuție al unei cereri

Pentru a înțelege integrarea completă a componentelor prezentate anterior, este utilă analiza fluxului de execuție al unei cereri tipice în aplicația *Kookio*, de la inițierea acesteia în browser până la accesul la datele persistente.

Atunci când un utilizator accesează o rută a aplicației, cererea HTTP este preluată de runtime-ul server-side furnizat de Analog, bazat pe infrastructura **Nitro** [6, 10]. În acest stadiu, aplicația Angular este executată pe server, iar structura inițială a paginii este randată sub formă de HTML [6].

În timpul randării server-side, componentele Angular pot declanșa apeluri către proceduri **tRPC**, definite în stratul server-side al aplicației [16]. Aceste proceduri reprezintă punctul de intrare pentru logica de business și realizează validarea datelor de intrare prin scheme Zod, generate din modelul Prisma [2, 15, 4].

Procedurile tRPC utilizează **Prisma Client** pentru a interacționa cu baza de date **CockroachDB**, executând operații de citire sau scriere într-un context tranzacțional sigur [15, 3]. Rezultatele interogărilor sunt returnate către nivelul tRPC, apoi propagate către componentele Angular care participă la randarea inițială a paginii [16].

După finalizarea procesului de randare server-side, HTML-ul rezultat este trimis către browser. În paralel, aplicația Angular este descărcată și inițializată pe client, unde are loc procesul de **hidratare** (*hydration*), prin care structura DOM generată pe server este reutilizată și îmbogățită cu interactivitate [6].

Apelurile ulterioare inițiate din interfața utilizatorului (de exemplu, adăugarea sau actualizarea unui ingredient) sunt realizate direct din browser către procedurile tRPC, fără a mai necesita o randare completă pe server [16]. Astfel, aplicația combină avantajele randării server-side pentru încărcarea inițială cu flexibilitatea și performanța interacțiunilor client-side.

Acest flux de execuție unitar demonstrează beneficiile arhitecturii full-stack adoptate în proiectul *Kookio*, în care frontend-ul, backend-ul și stratul de persistență funcționează coerent într-un singur ecosistem TypeScript [9, 6, 16, 15].

3.2.9 Testare și validare

Procesul de validare a aplicației include atât testare automată, cât și verificare manuală, fiind integrat în fluxul de dezvoltare al monorepo-ului Nx [9].

Pentru testarea unitară și de integrare este utilizat **Vitest**, ales pentru integrarea sa nativă cu Vite și compatibilitatea cu ecosistemul Angular utilizat de Analog [12, 11, 6]. Testele acoperă componentele UI, serviciile și logica de business, rulând într-un mediu controlat care permite izolarea dependențelor și verificarea comportamentului funcțiilor critice.

Testarea end-to-end este realizată cu **Playwright**, care simulează interacțiunea utilizatorului cu aplicația în browser [8]. Aceasta este deosebit de relevantă în contextul randării server-side (SSR), întrucât permite validarea fluxurilor complete, de la generarea inițială a paginii pe server până la interacțiunea cu datele persistente prin API-ul tRPC [6, 16].

Combinarea testelor unitare, de integrare și end-to-end contribuie la identificarea timpurie a defectelor și la creșterea stabilității aplicației, asigurând un comportament predictibil al sistemului în diferite scenarii de utilizare.

3.2.10 Dezvoltare și deploy

Aplicația este dezvoltată local folosind un mediu care reproduce cât mai fidel configurația de producție. Variabilele de mediu sunt gestionate centralizat prin platforma Vercel, iar pentru dezvoltarea locală acestea sunt sincronizate cu ajutorul Vercel CLI [18].

Deploy-ul aplicației este realizat pe platforma Vercel, care oferă suport nativ pentru aplicații server-side și integrare continuă. Această abordare permite rularea aplicației în medii distincte (dezvoltare, testare, producție), cu baze de date separate și configurări independente.

În practică, generarea Prisma Client trebuie realizată pe platforma țintă de execuție (Linux în cazul Vercel), deoarece binarele Prisma sunt dependente de sistemul de operare și de biblioteca SSL utilizată. Utilizarea unui client generat pe o mașină locală poate conduce la incompatibilități de tip `darwin-arm64` vs. `rhel-openssl-3.0.x`. Soluția adoptată constă în generarea Prisma Client în cadrul pipeline-ului de build și deploy.

Capitolul 4

Concluzii

Lucrarea de față a avut ca obiectiv proiectarea și implementarea unei aplicații web full-stack moderne, care să demonstreze integrarea coerentă a tehnologiilor actuale din ecosistemul TypeScript într-o arhitectură unitară. Proiectul *Kookio* a fost conceput atât ca aplicație funcțională, cât și ca studiu de caz pentru analiza arhitecturilor web hibride utilizate în contexte reale.

Prin adoptarea arhitecturii **PAN** (Prisma–Analog–Nx), aplicația reușește să asigure un flux end-to-end tipizat, de la interfața utilizatorului până la nivelul de persistență al datelor. Această abordare reduce discrepanțele dintre frontend și backend, elimină duplicarea contractelor de date și facilitează dezvoltarea incrementală a funcționalităților, poziționând arhitectura PAN ca un model viabil pentru aplicații web full-stack moderne.

Utilizarea **Angular** împreună cu **Analog** a permis implementarea unei aplicații cu randare server-side și hidratare client-side, îmbunătățind performanța percepută și compatibilitatea cu motoarele de căutare. Integrarea logicii de server în aceeași aplicație cu interfața utilizatorului a simplificat arhitectura generală și a redus necesitatea unui backend separat, fără a compromite separarea responsabilităților.

Pentru accesul la date, combinația dintre **Prisma** și **CockroachDB** oferă un model de persistență robust, compatibil cu standardele SQL, dar pregătit pentru scenarii de scalare viitoare. Definirea declarativă a schemei bazei de date și generarea automată a clientului TypeScript contribuie la creșterea siguranței, lizibilității și mentenabilității codului.

Procesul de testare a evidențiat importanța combinării mai multor niveluri de validare. Testele unitare și de integrare realizate cu **Vitest** asigură corectitudinea logicii interne, în timp ce testele end-to-end cu **Playwright** validează funcționarea aplicației din perspectiva utilizatorului final, inclusiv în contextul randării server-side.

Din punct de vedere al obiectivelor inițiale, proiectul îndeplinește cerințele funcționale ale unui MVP (Minimum Viable Product), oferind funcționalități de bază pentru gestionarea ingredientelor și demonstrând fezabilitatea arhitecturii propuse. Structura modulară a monorepo-ului și utilizarea Nx permit extinderea ulterioară a aplicației fără modificări structurale majore.

În concluzie, aplicația *Kookio* demonstrează că arhitectura PAN poate fi utilizată ca model coerent pentru dezvoltarea aplicațiilor web full-stack moderne, oferind atât valoare practică, cât și un cadru solid pentru analiză academică și dezvoltare ulterioară.

Direcții viitoare

Direcțiile viitoare de dezvoltare includ extinderea mecanismelor de autentificare și autorizare, optimizarea performanței prin introducerea unor strategii de caching și integrarea unor servicii inteligente pentru recomandarea rețetelor.

Aceste extensii pot fi realizate incremental, fără a afecta fundamentele arhitecturale prezentate în această lucrare.

Appendices

Anexa A

Anexă: Redactarea lucrării în L^AT_EX

A.1 Instrumente pentru redactare

Lucrarea a fost redactată cu motorul LuaLaTeX, folosind fonturi Unicode și pachetul `biblatex` pentru gestionarea bibliografiei. Structura este modulară, fiecare capitol fiind un fișier separat (`.tex`) compilat prin scripturile automate de build. Imaginile sunt salvate în `images/`, iar citările se realizează cu `\cite{}`.

A.2 Compatibilitate multiplatformă

Există scripturi automate pentru Linux (`build.sh`), Windows (`build.bat`) și PowerShell (`build.ps1`) care generează PDF-ul în directorul `dist/` și afișează eventualele erori de compilare.

Notă: Mediul `TreeVerb` utilizează fontul `FreeMono` pentru suport complet al caracterelor Unicode de tip box-drawing (U+2500–U+257F).

Anexa B

Kookio: Manifest Arhitectural PAN (Prisma – Analog – Nx)

B.1 Viziune generală

Kookio este o aplicație full-stack dezvoltată în **TypeScript**, bazată pe arhitectura **PAN** — un cadru scalabil care integrează **Prisma** pentru acces la date, **Analog** pentru randare hibridă și **Nx** pentru orchestrare monorepo. Această abordare reprezintă o alternativă la stack-uri consacrate (JAMstack, MEAN), fiind adaptată ecosistemului **Angular** și cerințelor enterprise.

B.2 Componentele arhitecturii

- **Prisma** – ORM cu suport pentru PostgreSQL și extensii NewSQL (CockroachDB).
- **Analog** – meta-framework pentru Angular cu SSR, Signals și tRPC.
- **Nx** – orchestrator modular care optimizează build-urile și testele.

B.3 Structura fizică a proiectului

Structura completă a monorepo-ului este ilustrată în § ?? din capitolul ??.

B.4 Avantaje comparative

Caracteristică	JAMstack	MEAN	PAN (Kookio)
SSR / Hybrid Rendering	Parțial (Next.js)	Nu	Da (Analog SSR)
Type-safety end-to-end	Parțial	Nu (Mongo/JS)	Da (TS + Prisma + Angular)
Structurare modulară	Slabă	Medie	Excelentă (Nx + libs)
Acces la date	Headless CMS/REST	REST/Mongoose	ORM modern (Prisma)
Adaptabilitate mono-repo	Nu	Nu	Da

Tabela B.1: Comparatie între arhitectura PAN și alte stack-uri moderne

B.5 Direcții de viitor

Direcțiile viitoare de dezvoltare sunt prezentate în § ??, unde sunt discutate extensiile funcționale și tehnice ale aplicației.

B.6 Concluzie

Arhitectura **PAN** sintetizează cele mai bune practici moderne din ecosistemul **TypeScript**, oferind o experiență unificată full-stack și o bază solidă pentru aplicații educaționale, alimentare sau de lifestyle, precum viziunea *Kookio*. **PAN** nu reprezintă un standard formal, ci un model arhitectural derivat din integrarea unor tehnologii mature, validat prin implementarea aplicației *Kookio*.

Bibliografie

- [1] Autorii Pino, *Pino — Logger pentru Node.js*, Documentația Pino, Documentație oficială Pino; citată pentru logarea centralizată utilizată atât în backend, cât și în frontend., 2025, URL: <https://getpino.io/> (accesat în 30.9.2025).
- [2] Autorii Zod, *Documentația Zod*, Documentația Zod, Documentație oficială Zod; citată pentru validarea schemelor utilizate împreună cu tipurile generate de Prisma., 2025, URL: <https://zod.dev> (accesat în 15.6.2025).
- [3] Cockroach Labs, *Documentația CockroachDB*, Documentația CockroachDB, Documentație oficială CockroachDB; citată în planul de dezvoltare pentru analiza scalabilității orizontale., 2025, URL: <https://www.cockroachlabs.com/docs/stable/> (accesat în 18.10.2025).
- [4] Omar Dulaimi și colaboratorii, *Generatorul Prisma Zod*, Depozitul GitHub, Versiunea 1.27.7; citată pentru generarea automată a schemelor de validare Zod pe baza modelelor Prisma., 2025, URL: <https://github.com/omar-dulaimi/prisma-zod-generator> (accesat în 18.10.2025).
- [5] Comunitatea JSON Schema, *JSON Schema — Învăță (Learn)*, Documentația JSON Schema, Resursă oficială JSON Schema; citată pentru validarea externă și interoperabilitatea menționată în planul de dezvoltare., 2025, URL: <https://json-schema.org/learn/> (accesat în 15.6.2025).
- [6] Dezvoltatorii AnalogJS, *Analog – Meta-framework full-stack pentru Angular*, Documentația Analog, Documentație oficială Analog; citată pentru SSR, rutare bazată pe fișiere și integrarea Nitro/tRPC în aplicațiile Angular., 2025, URL: <https://analogjs.org> (accesat în 14.9.2025).
- [7] Echipa Google Angular, *Prezentare generală a semnalelor (Signals Overview)*, Documentația Angular, Documentație oficială Angular; citată pentru modelul de reactivitate (Signals) utilizat în componente independente și în contextul randării pe server (SSR)., 2025, URL: <https://angular.dev/guide/signals> (accesat în 15.6.2025).
- [8] Echipa Microsoft Playwright, *Testarea cu Playwright — Ghid introductiv*, Documentația Playwright, Documentație oficială Playwright; citată pentru testarea end-to-end adoptată în perioada iunie–iulie 2025., 2025, URL: <https://playwright.dev/docs/intro> (accesat în 10.7.2025).
- [9] Echipa Nrwl, *Ce este Nx?*, Documentația Nx, Documentație oficială Nx; citată pentru orchestrarea monorepo-ului, rulările afectate, caching și graful de proiect., 2025, URL: <https://nx.dev/getting-started/intro> (accesat în 15.6.2025).

-
- [10] Echipa UnJS, *Nitro: Motorul de server pentru web*, Documentația Nitro, Documentație oficială Nitro; citată pentru execuția pe server, rutele API și mediile de implementare compatibile cu Analog SSR., 2025, URL: <https://nitro.unjs.io> (accesat în 14.9.2025).
 - [11] Echipa Vite, *Ghidul Vite*, Documentația Vite, Documentație oficială Vite; citată pentru configurarea build-ului comună aplicațiilor și bibliotecilor și integrarea cu țintele Nx., 2025, URL: <https://vitejs.dev/guide/> (accesat în 14.9.2025).
 - [12] Echipa Vitest, *Ghidul Vitest*, Documentația Vitest, Documentație oficială Vitest; citată pentru configurarea și utilizarea testelor unitare, folosită în proiect începând cu august 2025., 2025, URL: <https://vitest.dev/guide/> (accesat în 12.8.2025).
 - [13] Google Firebase, *Autentificarea în Firebase*, Documentația Firebase, Documentație oficială Firebase; citată pentru autentificarea planificată și politicile de autorizare bazate pe roluri., 2025, URL: <https://firebase.google.com/docs/auth> (accesat în 10.7.2025).
 - [14] Grupul Global de Dezvoltare PostgreSQL, *Documentația oficială PostgreSQL*, Documentația PostgreSQL, Documentație oficială PostgreSQL; citată pentru semantica SQL, suportul JSONB și siguranța tranzacțiilor utilizate prin Prisma., 2025, URL: <https://www.postgresql.org/docs/> (accesat în 18.10.2025).
 - [15] Prisma Labs, *Documentația Prisma ORM*, Documentația Prisma, Documentație oficială Prisma; citată pentru modelarea schemei, migrații, generarea clientului TypeScript și integrarea cu Zod., 2025, URL: <https://www.prisma.io/docs> (accesat în 18.10.2025).
 - [16] Proiectul tRPC, *Documentația tRPC — API-uri type-safe end-to-end*, Documentația tRPC, Documentație oficială tRPC; citată pentru contractele de tipuri între client și server și pentru structura router/context., 2025, URL: <https://trpc.io/docs> (accesat în 14.9.2025).
 - [17] Redis, *Documentația Redis*, Documentația Redis, Documentație oficială Redis; citată în planul de dezvoltare pentru strategiile de caching și gestionare a sesiunilor de interogare., 2025, URL: <https://redis.io/docs/latest/> (accesat în 20.10.2025).
 - [18] Vercel, *Vercel CLI — Command Line Interface*, Documentația Vercel, Documentație oficială pentru gestionarea proiectelor și a variabilelor de mediu în Vercel., 2025, URL: <https://vercel.com/docs/cli> (accesat în 16.12.2025).